OOP for Competitive Programming — Revision Notes

Phases 1–13

Piyush | IIIT Kota

oday

Contents

1 Phase 1: OOP Foundations

1.1 What is OOP?

Programming paradigm organized around **objects** (data + behavior) instead of just functions acting on data.

- Four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.
- Why it matters for CP: building reusable, bug-resistant utility classes (DSU, Segment Tree, modint) instead of copy-pasting global arrays/functions every contest.

1.2 Objects and Classes

- **Class** = blueprint (type). **Object** = instance (actual memory).

```
class Point {
public:
    int x, y;
};
Point p1;        // stack object
Point* p2 = new Point(); // heap object
```

- `sizeof(class)` = sum of members (+ padding); empty class has size 1 byte.

1.3 Access Specifiers

Specifier	Class members	Derived class	Outside
`public`	yes	yes	yes
`protected`	yes	yes	no
`private`	yes	no	no

- `class` defaults to `private`, `struct` defaults to `public`. Only real difference between the two in C++.

1.4 Constructors

- Special member function, same name as class, no return type, runs on object creation.

```
class Fenwick {
public:
    vector<int> tree;
    Fenwick(int n) : tree(n + 1, 0) {}   // constructor
};
```

- Compiler auto-generates a default constructor **only if you define no other constructor**.
- Constructors can be overloaded (see Phase 3).

1.5 Destructor

- `~ClassName()`, no args, no overloading, runs automatically at scope exit / `delete`.

```
class Graph {
public:
    int* adjMatrix;
```

```
    Graph(int n) { adjMatrix = new int[n * n]; }
    ~Graph() { delete[] adjMatrix; }   // free heap memory
};
```

- Critical when class manages raw pointers/heap memory, otherwise memory leaks.
- Virtual destructor needed in polymorphic base classes (Phase 8).

1.6 `this` Pointer

- Implicit pointer to the calling object, available inside non-static member functions.
- Used to disambiguate member vs parameter with same name, and for method chaining.

```
class Node {
    int val;
public:
    Node& setVal(int val) { this->val = val; return *this; } // chaining
};
```

2 Phase 2: Encapsulation

2.1 Encapsulation

- Bundling data + methods together, restricting direct access to internal state via `private/protected`.
- Goal: invariants stay valid, object controls how its own data changes.

2.2 Getters & Setters

```
class BankAccount {
    double balance;
public:
    double getBalance() const { return balance; }
    void setBalance(double b) {
        if (b < 0) throw invalid_argument("negative balance");
        balance = b;
    }
};
```

- Setter can validate input; plain public member cannot.

2.3 `const` Member Functions

- Promise not to modify object state; required to call on `const` objects/references.

```
int getX() const { return x; }   // cannot modify members here
```

- Pass large objects as `const Type&` to avoid copies while keeping safety, very common in CP utility classes.

2.4 Mutable Members

- `mutable` keyword lets a member be modified even inside a `const` function.

```
class Cache {
    mutable int callCount = 0;
public:
    int compute() const { callCount++; return 42; } // allowed
};
```

- Typical use: internal counters, memoization caches, lazy-init fields.

2.5 Why Encapsulation Matters

- Prevents invalid states (e.g., negative array size).
- Decouples internal representation from external interface, you can change implementation without breaking callers.
- In CP: fewer silent bugs when a teammate/you misuse an internal array directly.

3 Phase 3: Constructors Deep Dive

3.1 Default Constructor

- Takes no arguments (or all defaulted). Compiler-generated one default-initializes members (primitives left **uninitialized**, not zero).

```
class DSU {
    vector<int> par;
public:
    DSU() {}          // user-defined default ctor
};
```

3.2 Parameterized Constructor

```
class DSU {
    vector<int> par, rnk;
public:
    DSU(int n) : par(n), rnk(n, 0) {
        iota(par.begin(), par.end(), 0);
    }
};
```

3.3 Constructor Overloading

- Multiple constructors, different parameter lists (like function overloading).

```
class Matrix {
public:
    Matrix() {}                          // 1
    Matrix(int n) { /* n x n */ }        // 2
    Matrix(int r, int c) { /* r x c */ } // 3
};
```

3.4 Copy Constructor

- `ClassName(const ClassName& other)`. Called on pass-by-value, return-by-value (sometimes elided), and explicit copy.

```
class Buffer {
    int* data; int n;
public:
    Buffer(int n) : n(n) { data = new int[n]; }
    Buffer(const Buffer& o) : n(o.n) {      // deep copy
        data = new int[n];
        copy(o.data, o.data + n, data);
    }
};
```

- Compiler default copy ctor does **shallow copy**, dangerous with raw pointers (Phase 4).

3.5 Initializer List

```cpp
class Point {
    const int x, y;   // const members MUST use init list
public:
    Point(int x, int y) : x(x), y(y) {}
};
```

- Required for `const` members, reference members, and base-class subobjects without default ctors.
- Members initialize in **declaration order**, not init-list order, common CP interview gotcha.

3.6 Delegating Constructors

```cpp
class Rect {
    int w, h;
public:
    Rect(int w, int h) : w(w), h(h) {}
    Rect(int side) : Rect(side, side) {}   // delegates to above
};
```

3.7 Explicit Constructor

- `explicit` blocks implicit single-argument conversions.

```cpp
class BigInt {
public:
    explicit BigInt(long long v) {}
};
// BigInt b = 5;   // ERROR with explicit — forces BigInt b(5);
```

- Good default habit for single-arg constructors to avoid silent, surprising conversions.

3.8 Deleted Constructors

```cpp
class Singleton {
public:
    Singleton() = default;
    Singleton(const Singleton&) = delete;   // no copies allowed
};
```

- `= delete` explicitly forbids a function overload from being used (also used for Rule of Three enforcement, Phase 4).

4 Phase 4: Memory — Stack, Heap and Copy Semantics

4.1 Stack vs Heap Objects

	Stack	Heap
Alloc	automatic, fast	manual, slower
Lifetime	scope-bound	until `delete`
Size limit	small (~MBs)	large
CP use	default choice	huge arrays / size unknown at compile time

```
Point p;              // stack — destroyed at end of scope
Point* q = new Point(); // heap — must delete q manually
```

4.2 `new` and `delete`

```
int* arr = new int[n];       // allocate
delete[] arr;                 // must match with delete[] for arrays
int* x = new int(5);
delete x;                     // single object -> plain delete
```

- Mismatching `new[]`/`delete` is undefined behavior. Forgetting `delete` = memory leak (usually fine in short-lived CP programs, but bad habit).

4.3 Shallow Copy

- Default copy ctor/assignment copies pointer **values**, not what they point to, so two objects share one buffer, causing double-free / dangling pointer bugs.

4.4 Deep Copy

- Manually copy pointed-to data so each object owns independent memory (see copy ctor example, Phase 3).

4.5 Copy Assignment Operator

```
Buffer& operator=(const Buffer& o) {
    if (this == &o) return *this;      // self-assignment guard
    delete[] data;
    n = o.n;
    data = new int[n];
    copy(o.data, o.data + n, data);
    return *this;
}
```

4.6 Rule of Three

- If you write **any one** of: destructor, copy constructor, copy assignment operator, you almost certainly need **all three**, because the class is managing a resource (raw pointer) the compiler-generated versions would mishandle.

4.7 Rule of Five (overview)

- C++11 adds **move constructor** and **move assignment operator** to the Rule of Three, for efficient transfer of resources without deep copy when the source is a temporary/rvalue.

4.8 Move Constructor (basic)

```
Buffer(Buffer&& o) noexcept : data(o.data), n(o.n) {
    o.data = nullptr;   // steal, don't copy
    o.n = 0;
}
```

- Triggered on rvalues (temporaries, `std::move(x)`). O(1) “steal” instead of O(n) deep copy, matters for large CP structures passed around.

5 Phase 5: Static, Friend and Anonymous Objects

5.1 Static Data Members

- One copy shared across **all** objects of the class; declared in class, defined outside (pre-C++17) or `inline static` (C++17+).

```cpp
class Counter {
public:
    static int count;
    Counter() { count++; }
};
int Counter::count = 0;   // definition outside class
```

5.2 Static Member Functions

- Can only access static members; called without an object: `ClassName::func()`.

```cpp
class MathUtil {
public:
    static long long modpow(long long b, long long e, long long m) {
        long long r = 1; b %= m;
        while (e) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
        return r;
    }
};
// MathUtil::modpow(2, 10, 1e9+7);
```

- Very common pattern for CP “utility/math class” (Phase 12).

5.3 Friend Functions

- Non-member function granted access to private/protected members.

```cpp
class Vec2 {
    int x, y;
public:
    Vec2(int x, int y): x(x), y(y) {}
    friend Vec2 operator+(const Vec2&, const Vec2&);
};
Vec2 operator+(const Vec2& a, const Vec2& b) { return Vec2(a.x+b.x, a.y+b.y); }
```

5.4 Friend Classes

```cpp
class Engine;
class Car {
    friend class Engine;   // Engine can access Car's private members
    int horsepower = 0;
};
```

5.5 Anonymous Objects

- Temporary object with no named variable, used inline and destroyed at end of full expression.

```
cout << Point(3, 4).x;   // Point(3,4) is anonymous
process(Matrix(5));      // pass temporary directly
```

6 Phase 6: Operator Overloading

6.1 Operator Overloading Basics

- Give class-defined meaning to built-in operators, either as **member function** (`a.operator+(b)`) or **free/friend function**.
- Cannot overload: `.`, `::`, `?:`, `sizeof`.

6.2 Binary Operators

```
struct BigNum {
    vector<int> d;
    BigNum operator+(const BigNum& o) const {
        BigNum r; /* addition logic */ return r;
    }
};
```

6.3 Unary Operators

```
struct Vec2 {
    int x, y;
    Vec2 operator-() const { return {-x, -y}; }   // unary minus
};
```

6.4 Comparison Operators

```
struct Point {
    int x, y;
    bool operator<(const Point& o) const {   // needed for set/map/sort
        return x != o.x ? x < o.x : y < o.y;
    }
    bool operator==(const Point& o) const { return x == o.x && y == o.y; }
};
```

- C++20 lets you write just `operator<=>` (spaceship) to auto-generate the rest, good to know exists, but most CP judges still run C++17.

6.5 Stream Operators (<< >>)

- Must be free functions (LHS is `ostream`, not your class), usually `friend`.

```
struct Frac {
    int num, den;
    friend ostream& operator<<(ostream& os, const Frac& f) {
        return os << f.num << "/" << f.den;
    }
};
```

6.6 [ ] Operator

```cpp
class Matrix {
    vector<vector<int>> m;
public:
    vector<int>& operator[](int i) { return m[i]; }   // enables mat[i][j]
};
```

6.7 () Operator (functor)

```cpp
struct CompareByFreq {
    bool operator()(int a, int b) const { return freq[a] > freq[b]; }
};
// priority_queue<int, vector<int>, CompareByFreq> pq;
```

6.8 Practical CP Examples

- `operator<` on struct: drop directly into `sort()`, `set`, `map`, `priority_queue`.
- `operator()` functor: custom comparator without a global function.
- `operator<<`: one-line debug printing for custom structs.

7 Phase 7: Inheritance

7.1 Base and Derived Class

```cpp
class Shape {           // base
public:
    double area() { return 0; }
};
class Circle : public Shape {   // derived
    double r;
public:
    Circle(double r): r(r) {}
};
```

7.2 Types of Inheritance

- Single, Multilevel, Multiple, Hierarchical, Hybrid.

```
Single:       A -> B
Multilevel:   A -> B -> C
Hierarchical: A -> B, A -> C
Multiple:     A, B -> C
```

7.3 Access Modes

Base member	`public` inheritance	`protected` inheritance	`private` inheritance
public	public	protected	private
protected	protected	protected	private
private	inaccessible	inaccessible	inaccessible

- CP convention: almost always `public` inheritance.

7.4 Constructor Order

- Base class constructor runs **before** derived class constructor body.

```cpp
class Base { public: Base() { cout << "Base\n"; } };
class Derived : public Base { public: Derived() { cout << "Derived\n"; } };
// output: Base then Derived
```

7.5 Destructor Order

- Exact reverse: derived destructor runs first, then base destructor.

7.6 Protected Keyword

- Accessible in the class itself **and** derived classes, but not from outside, used for fields sub-classes need to extend/modify directly.

7.7 Multiple Inheritance

```
class Printable { public: void print() {} };
class Comparable { public: bool cmp() { return true; } };
class Item : public Printable, public Comparable {};
```

- Watch for the **diamond problem** (two base classes sharing a common ancestor), solved with `virtual` inheritance, rarely needed in CP.

8 Phase 8: Polymorphism

8.1 Function Overloading

- Same function name, different parameter list, resolved at **compile time**.

```
int add(int a, int b);
double add(double a, double b);
```

8.2 Function Overriding

- Derived class redefines a base class's `virtual` function with the **same signature**, resolved at **runtime**.

```
class Shape { public: virtual double area() { return 0; } };
class Square : public Shape {
    double side;
public:
    double area() override { return side * side; }
};
```

8.3 Virtual Functions

- Declared `virtual` in base, enables **dynamic dispatch**: call resolved based on actual object type, not pointer/reference's static type.

```
Shape* s = new Square(5);
s->area();   // calls Square::area(), not Shape::area()
```

8.4 Virtual Table (vtable)

- Each polymorphic class gets a hidden static array of function pointers (vtable); each object holds a hidden `vptr` to its class's vtable.
- Virtual calls cost one extra pointer indirection, negligible for CP unless in an extremely hot loop.

8.5 Pure Virtual Functions

```
class Shape {
public:
    virtual double area() = 0;   // pure virtual — no body required
};
```

8.6 Abstract Classes

- Any class with at least one pure virtual function; **cannot be instantiated**, only used as a base/interface.

8.7 Dynamic Binding

- Function call resolved at runtime via vtable lookup (as opposed to **static binding**, resolved at compile time, normal functions, overloads).

8.8 Object Slicing

```
Square sq(5);
Shape s = sq;      // SLICED: only Shape part copied, Square-specific data lost
s.area();          // calls Shape::area(), NOT Square::area() — bug!
```

- Fix: always use Shape* or Shape& (pointer/reference) for polymorphic behavior, never pass/store by value.
- Rule of thumb: polymorphic base class, give it a `virtual ~Shape() {}` destructor too, or deleting via base pointer is UB.

9 Phase 9: Generic Programming and Modern C++ Utilities

9.1 Templates (Function)

```cpp
template <typename T>
T maxVal(T a, T b) { return a > b ? a : b; }
// maxVal<int>(3, 5);
// maxVal(3.5, 2.1);   // type deduced
```

9.2 Class Templates

```cpp
template <typename T>
class Stack {
    vector<T> data;
public:
    void push(T x) { data.push_back(x); }
    T pop() { T x = data.back(); data.pop_back(); return x; }
};
// Stack<int> s1; Stack<string> s2;
```

- This is exactly how you'd generalize a Fenwick Tree / Segment Tree to work over `int`, `long long`, or a custom `struct` (Phase 13).

9.3 Template Specialization

```cpp
template <typename T>
class Printer { public: void print(T v) { cout << v; } };

template <>                          // full specialization
class Printer<bool> {
public: void print(bool v) { cout << (v ? "true" : "false"); }
};
```

9.4 auto

```cpp
auto it = mp.begin();          // deduces map<...>::iterator
auto sum = 0LL;                // deduces long long from literal
```

9.5 decltype

```cpp
int x = 5;
decltype(x) y = 10;            // y is int
decltype(x + 1.0) z;           // z is double
```

9.6 constexpr

```cpp
constexpr int MOD = 1e9 + 7;         // evaluated at compile time
constexpr long long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
```

- Great for CP constants and small compile-time lookup tables.

9.7 inline

- Suggests compiler avoid function-call overhead by substituting the body at call site; **required** for functions defined in headers included in multiple translation units (rare concern in single-file CP, common in template-library headers).

9.8 namespace

```
namespace geometry {
    struct Point { int x, y; };
}
// geometry::Point p;
// using namespace geometry;   // brings names into scope, avoid in headers
```

9.9 Type Aliases (using)

```
using ll = long long;
using pii = pair<int, int>;
using Graph = vector<vector<int>>;
template <typename T> using vec2 = vector<vector<T>>;   // alias template
```

10 Phase 10: STL Internals for CP

10.1 `vector` Internals

- Contiguous dynamic array; grows by **doubling** capacity on overflow (amortized O(1) push_back).
- `reserve(n)` pre-allocates to avoid repeated reallocation/copy, use when final size is known.
- `.size()` = element count, `.capacity()` = allocated slots (capacity >= size).

10.2 `string` Class

- Essentially `vector<char>` + null-terminator compatibility (`c_str()`); Small String Optimization (SSO) avoids heap alloc for short strings (implementation-defined, roughly 15-22 chars).

10.3 `pair`

```
pair<int, int> p = {1, 2};
// p.first; p.second;
// auto [a, b] = p;   // structured bindings, C++17
```

- Has built-in lexicographic `operator<`, sorts by `.first` then `.second`.

10.4 `tuple`

```
tuple<int, string, double> t = {1, "hi", 3.5};
// get<0>(t); get<1>(t);
// auto [id, name, val] = t;
```

10.5 `priority_queue`

```
priority_queue<int> maxHeap;                              // max-heap default
priority_queue<int, vector<int>, greater<int>> minHeap;    // min-heap
```

- Backed by `vector` + heap operations, O(log n) push/pop, O(1) top.

10.6 `queue`

- FIFO adapter, default backed by deque; O(1) push/pop/front.

10.7 `stack`

- LIFO adapter, default backed by deque; O(1) push/pop/top.

10.8 `map`

- Red-Black tree (balanced BST), sorted by key, O(log n) insert/find/erase.

```
map<int, int> mp;
mp[5] = 10;                 // insert-or-update
// if (mp.count(5)) { }        // existence check
```

10.9 unordered_map

- Hash table, average O(1) ops but O(n) worst case. **CP gotcha**: vulnerable to anti-hash test cases against unordered_map<int,...>, mitigate with a custom hash (splitmix64) or just use map/gp_hash_table.

10.10 set

- Sorted unique elements, Red-Black tree, O(log n) insert/find/erase; supports lower_bound/upper_bound in O(log n), unordered_set does not.

10.11 unordered_set

- Hash table version of set; O(1) average, no ordering, no lower_bound.

11 Phase 11: Function Objects and Lambdas

11.1 Function Objects

- A class/struct that overloads `operator()`, so its instances can be "called" like functions but can also carry state, see Phase 6.

11.2 operator()

```
struct Adder {
    int base;
    Adder(int b): base(b) {}
    int operator()(int x) const { return x + base; }
};
// Adder add5(5);
// add5(10);   // 15
```

11.3 Custom Comparators

```
struct byLength {
    bool operator()(const string& a, const string& b) const {
        return a.size() < b.size();
    }
};
// sort(v.begin(), v.end(), byLength());
// set<string, byLength> s;
```

11.4 Lambda Functions

```
auto sq = [](int x) { return x * x; };
// sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

- Syntax: `[captures](params) -> returnType { body }`. Return type usually deducible, omit it.

11.5 Capture Lists

```
int k = 5;
auto byCap  = [k](int x) { return x + k; };     // capture by value (copy)
auto byRef  = [&k](int x) { return x + k; };    // capture by reference
auto both   = [=]() { };                        // capture all by value
auto allRef = [&]() { };                        // capture all by reference
```

- Common CP use: custom `sort`/`priority_queue` comparator capturing an outer array by reference.

```
vector<int> a = {5,2,8,1};
vector<int> idx = {0,1,2,3};
sort(idx.begin(), idx.end(), [&](int i, int j) { return a[i] < a[j]; }); // sort indices by
```

11.6 `std::function` (basic)

```cpp
function<int(int,int)> op = [](int a, int b) { return a + b; };
// recursive lambda needs std::function or explicit self-capture:
function<void(int)> dfs = [&](int u) { for (int v : adj[u]) dfs(v); };
```

- More overhead than a raw lambda/template callback (type erasure), fine for CP, avoid in the tightest inner loops.

12 Phase 12: Writing Reusable Utility Classes

12.1 Writing Utility Classes

- Goal: wrap a repeated CP pattern (mod arithmetic, fast IO, debugging) into a tested, drop-in class you paste once per contest.

12.2 Namespace vs Class

- `namespace`: grouping of free functions/constants, no access control, no instances.
- `class`: encapsulation + access control + can be instantiated/templated.
- Rule of thumb: stateless helpers (constants, math free functions) go in a `namespace`; stateful structures (DSU, segment tree) go in a `class`.

12.3 Struct vs Class

- Functionally identical except default access (`struct` = public, `class` = private) and default inheritance mode.
- CP convention: `struct` for POD-like/CP utility types (less boilerplate, no `public:` needed); `class` when you want to enforce encapsulation intentionally.

12.4 Header-only Style

- Put full implementation in a `.hpp`, `#include` directly into your CP template, matches how your existing graph-algorithm template library is organized.

```
#pragma once   // include guard, avoids double-inclusion errors
```

12.5 Fast IO Class

```
struct FastIO {
    FastIO() {
        ios_base::sync_with_stdio(false);
        cin.tie(nullptr);
    }
} fastio_init;   // global instance runs setup before main()
```

12.6 Modular Arithmetic Class

```
template <int MOD>
struct ModInt {
    long long val;
    ModInt(long long v = 0) { val = ((v % MOD) + MOD) % MOD; }
    ModInt operator+(const ModInt& o) const { return ModInt(val + o.val); }
    ModInt operator*(const ModInt& o) const { return ModInt(val * o.val); }
    ModInt pow(long long e) const {
        ModInt r(1), b(*this);
        while (e) { if (e & 1) r = r * b; b = b * b; e >>= 1; }
        return r;
    }
    ModInt inv() const { return pow(MOD - 2); }   // MOD must be prime
```

```
};
using mint = ModInt<1000000007>;
```

12.7 Matrix Class

```
struct Matrix {
    vector<vector<long long>> a;
    int n;
    Matrix(int n): n(n), a(n, vector<long long>(n, 0)) {}
    Matrix operator*(const Matrix& o) const {
        Matrix r(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++) if (a[i][k])
                for (int j = 0; j < n; j++)
                    r.a[i][j] = (r.a[i][j] + a[i][k] * o.a[k][j]) % 1000000007;
        return r;
    }
};
```

12.8 Debug Class

```
#ifdef LOCAL
#define debug(x) cerr << #x << " = " << (x) << endl
#else
#define debug(x)
#endif
```

- A debug macro that's stripped in submission builds but active locally, pairs well with your VS Code Code Runner / CPH setup.

13 Phase 13: Data Structures as Classes

13.1 Fenwick Tree

```
struct Fenwick {
    vector<long long> t; int n;
    Fenwick(int n): n(n), t(n + 1, 0) {}
    void update(int i, long long v) { for (++i; i <= n; i += i & -i) t[i] += v; }
    long long query(int i) { long long s = 0; for (++i; i > 0; i -= i & -i) s += t[i]; retu
    long long range(int l, int r) { return query(r) - (l ? query(l - 1) : 0); }
};
```

13.2 Segment Tree

```
struct SegTree {
    int n; vector<long long> tree;
    SegTree(int n): n(n), tree(4 * n, 0) {}
    void build(vector<int>& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int mid = (l + r) / 2;
        build(a, 2*node, l, mid); build(a, 2*node+1, mid+1, r);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
    long long query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr);
    }
};
```

13.3 Lazy Segment Tree

- Adds a `lazy[]` array; range updates get deferred ("pushed down") to children only when a query/update actually descends into them, turns O(n) range update into O(log n).

```
void push_down(int node) {
    if (lazy[node]) {
        for (int c : {2*node, 2*node+1}) {
            tree[c] += lazy[node]; lazy[c] += lazy[node];
        }
        lazy[node] = 0;
    }
}
```

13.4 Sparse Table

- Static (no updates) range-query structure for idempotent ops (min/max/gcd), O(1) query after O(n log n) build.

```
struct SparseTable {
    vector<vector<int>> table;
    SparseTable(vector<int>& a) {
```

```
        int n = a.size(), LOG = log2(n) + 1;
        table.assign(LOG, vector<int>(n));
        table[0] = a;
        for (int j = 1; j < LOG; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                table[j][i] = min(table[j-1][i], table[j-1][i + (1 << (j-1))]);
    }
    int query(int l, int r) {
        int j = log2(r - l + 1);
        return min(table[j][l], table[j][r - (1 << j) + 1]);
    }
};
```

13.5 Trie

```
struct Trie {
    struct Node { Node* child[26] = {}; bool end = false; };
    Node* root = new Node();
    void insert(const string& s) {
        Node* cur = root;
        for (char c : s) {
            if (!cur->child[c-'a']) cur->child[c-'a'] = new Node();
            cur = cur->child[c-'a'];
        }
        cur->end = true;
    }
};
```

13.6 DSU

```
struct DSU {
    vector<int> par, rnk;
    DSU(int n): par(n), rnk(n, 0) { iota(par.begin(), par.end(), 0); }
    int find(int x) { return par[x] == x ? x : par[x] = find(par[x]); }  // path compressio
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rnk[a] < rnk[b]) swap(a, b);
        par[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return true;   // union by rank
    }
};
```

13.7 Graph Class

```
struct Graph {
    int n; vector<vector<int>> adj;
    Graph(int n): n(n), adj(n) {}
    void addEdge(int u, int v, bool directed = false) {
        adj[u].push_back(v);
```

```
        if (!directed) adj[v].push_back(u);
    }
};
```

13.8 Dijkstra Class

```
struct Dijkstra {
    vector<vector<pair<int,int>>> adj;  // {to, weight}
    vector<long long> dist;
    Dijkstra(int n): adj(n), dist(n, LLONG_MAX) {}
    void addEdge(int u, int v, int w) { adj[u].push_back({v, w}); }
    void run(int src) {
        dist[src] = 0;
        priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
        pq.push({0, src});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (auto [v, w] : adj[u])
                if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
        }
    }
};
```

13.9 LCA Class

- Encapsulates binary-lifting table (`up[k][v]`) + depth array; O(n log n) preprocess, O(log n) query.

```
struct LCA {
    int LOG; vector<vector<int>> up; vector<int> depth;
    LCA(int n): LOG(log2(n) + 1), up(LOG, vector<int>(n)), depth(n) {}
    // build(): fill up[0][v] = parent[v] via DFS, then up[k][v] = up[k-1][up[k-1][v]]
    int query(int u, int v) { /* lift deeper node, then binary-search common ancestor */ re
};
```

13.10 SCC Class

- Wraps **Kosaraju** (2 DFS passes + reversed graph) or **Tarjan** (single DFS + low-link array) behind an `addEdge()` / `run()` interface, exposing `component[v]` at the end.

13.11 Max Flow Class

```
struct Dinic {
    struct Edge { int to; long long cap; };
    vector<Edge> edges; vector<vector<int>> adj;
    vector<int> level, it;
    Dinic(int n): adj(n), level(n), it(n) {}
    void addEdge(int u, int v, long long cap) {
        adj[u].push_back(edges.size()); edges.push_back({v, cap});
        adj[v].push_back(edges.size()); edges.push_back({u, 0});
    }
```

```
    // bfs() builds level graph, dfs() sends blocking flow; run() loops both
};
```

13.12 Sieve Class

```
struct Sieve {
    vector<bool> isComposite;
    vector<int> primes;
    Sieve(int n): isComposite(n + 1, false) {
        for (int i = 2; i <= n; i++) {
            if (!isComposite[i]) primes.push_back(i);
            for (int p : primes) {
                if ((long long)i * p > n) break;
                isComposite[i * p] = true;
                if (i % p == 0) break;   // linear sieve
            }
        }
    }
};
```

13.13 Combinatorics Class

```
struct Comb {
    vector<long long> fact, inv_fact;
    static const long long MOD = 1e9 + 7;
    Comb(int n): fact(n + 1), inv_fact(n + 1) {
        fact[0] = 1;
        for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
        inv_fact[n] = modpow(fact[n], MOD - 2, MOD);
        for (int i = n; i > 0; i--) inv_fact[i-1] = inv_fact[i] * i % MOD;
    }
    long long C(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
    }
    static long long modpow(long long b, long long e, long long m) {
        long long r = 1; b %= m;
        while (e) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
        return r;
    }
};
```

13.14 Matrix Exponentiation Class

```
struct MatrixExp {
    static Matrix power(Matrix base, long long e) {
        Matrix result(base.n);
        for (int i = 0; i < base.n; i++) result.a[i][i] = 1;   // identity
        while (e) {
            if (e & 1) result = result * base;
            base = base * base;
```

```
            e >>= 1;
        }
        return result;   // O(n^3 log e) — for linear recurrences (fib, tilings, etc.)
    }
};
```

13.15 Rolling Hash Class

```
struct RollingHash {
    vector<long long> h, p;
    long long MOD = 1e9 + 7, B = 131;
    RollingHash(const string& s) {
        int n = s.size(); h.assign(n + 1, 0); p.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i+1] = (h[i] * B + s[i]) % MOD;
            p[i+1] = (p[i] * B) % MOD;
        }
    }
    long long get(int l, int r) {   // hash of s[l..r], inclusive
        return ((h[r+1] - h[l] * p[r-l+1]) % MOD + MOD) % MOD;
    }
};
```

- Use double hashing (two different MOD/B pairs) in practice, single hash is exploitable by anti-hash tests, same issue as unordered_map.

14 Quick Revision Checklist

- Phase 1-3: Can you write a class with a parameterized ctor + init list from memory, no reference?
- Phase 4: Can you explain why a raw-pointer class needs Rule of Three without looking it up?
- Phase 6: Can you overload < and << on a struct and drop it into `sort`/`set`/`cout` immediately?
- Phase 8: Can you explain object slicing and why polymorphic classes need a virtual destructor?
- Phase 9-11: Comfortable writing a templated class + a comparator lambda with capture-by-reference?
- Phase 12-13: Could you rebuild your `ModInt`, `DSU`, and `Fenwick` classes from scratch, cold, in under 10 minutes each?